

華中科技大學

課程實驗報告

課程名稱： 自然語言處理實驗

專業班級： 計算機科學與技術 202001

學 號： U202215450

姓 名： 朱昊天

指導教師： 辜希武

報告日期： 2025 年 5 月 15 日

計算機科學與技術學院

目 录

1 中文分词实现.....	2
1.1 问题描述	2
1.2 基础模块	2
1.2.1 基于词典匹配的分词算法.....	2
1.2.2 基于统计学习的分词算法（Bi-LSTM+CRF）	2
1.3 系统实现	3
1.3.1 词典匹配算法实现.....	3
1.3.2 Bi-LSTM+CRF 模型实现	4
1.4 实验小结	5
2 中文命名实体识别实现.....	6
2.1 问题描述	6
2.2 基础模块	6
2.2.1 数据集与标注.....	6
2.2.2 模型架构.....	6
2.3 系统实现	6
2.3.1 数据预处理.....	6
2.3.2 模型构建与训练（model.py、run.py）	7
2.3.3 模型推断（infer.py）	8
2.3.4 分词优化融合策略.....	9
2.4 实验小结	9

1 中文分词实现

1.1 问题描述

中文分词是将连续的汉字序列切分为具有语义的词语序列的过程。由于中文词语之间没有显式分隔符，需通过算法识别词边界。本实验要求实现基于词典匹配和基于统计学习的分词算法，并对比不同算法的效果与性能，同时探索优化策略以提升分词准确率。

基础任务：实现正向最大匹配（FMM）、逆向最大匹配（BMM）算法，以及基于 Bi-LSTM+CRF 的统计学习分词模型。

选做任务：通过引入预训练模型、优化词典、融合命名实体识别结果等方式提升分词性能，基于测试集的 F1-score 评估优化效果。

1.2 基础模块

1.2.1 基于词典匹配的分词算法

正向最大匹配法（FMM）

流程：从左到右取句子前 m 个字（ m 为词典最长词长）匹配词典，成功则切分，否则去掉最后一个字重复匹配，直至完成分词。

示例：对句子“中国科学院”，若词典包含“中国”“科学院”，则切分为“中国 / 科学院”。

逆向最大匹配法（BMM）：从右到左取末端 m 个字匹配，逻辑与 FMM 类似，切分顺序相反。

双向最大匹配法：结合 FMM 和 BMM 结果，选择词数少或单字少的方案以减少歧义。

1.2.2 基于统计学习的分词算法（Bi-LSTM+CRF）

序列标注模型：采用“BEMS”标注体系（B - 词首，E - 词尾，M - 词中，S - 单字词），将分词转化为序列标注任务。

模型结构

嵌入层：将汉字转换为 100 维词向量（embedding_dim=100）。

Bi-LSTM 层：双向长短期记忆网络，隐藏层维度 200（hidden_dim=200），捕捉上下文语义。

CRF 层：条件随机场，优化标注序列的全局依赖关系，输出最终标签。

训练配置：使用 Adam 优化器，学习率 0.005，batch size 128，训练 10 轮，支持 CPU/GPU 计算。

1.3 系统实现

1.3.1 词典匹配算法实现

数据准备：使用实验提供的基础词典（Dictionary_based 文件夹），包含常用词约 5000 条，最长词长 4 字。

```
def fmm_segment(sentence, dictionary, max_len):
    words = []
    start = 0
    while start < len(sentence):
        end = start + max_len
        if end > len(sentence):
            end = len(sentence)
        while end > start:
            word = sentence[start:end]
            if word in dictionary:
                words.append(word)
                start = end
                break
            end -= 1
        else:
            words.append(sentence[start])
            start += 1
    return words
```

测试结果：对测试样本“小明在中科院工作”，FMM 切分为“小明 / 在 / 中科院 / 工作”，BMM 切分为“小 / 明 / 在 / 中科院 / 工作”，双向匹配选择 FMM 结果（词数更少）。

1.3.2 Bi-LSTM+CRF 模型实现

嵌入层 (Embedding):

输入: 字 ID 序列 [batch_size, seq_len]

输出: 字向量序列 [batch_size, seq_len, embedding_dim]

未登录字处理: 自动映射到 vocab_size 索引

双向 LSTM 层:

隐藏层维度: hidden_dim//2 (双向拼接后为 hidden_dim)

输出: 上下文特征 [batch_size, seq_len, hidden_dim]

线性映射层:

将 LSTM 特征投影到标签空间

输出: 标签分数 [batch_size, seq_len, 4]

CRF 层:

解码最优标签序列

计算全局规范化的负对数似然损失

数据加载与批处理相关代码:

```
class Sentence(Dataset):
    def __init__(self, x, y):
        self.x = x # 字 ID 序列
        self.y = y # 标签 ID 序列

    def __getitem__(self, idx):
        return self.x[idx], self.y[idx]

    @staticmethod
    def collate_fn(batch):
        # 填充序列到相同长度
        x_batch, y_batch = zip(*batch)
        max_len = max(len(x) for x in x_batch)
        x_padded = [x + [0]*(max_len-len(x)) for x in x_batch]
        y_padded = [y + [-1]*(max_len-len(y)) for y in y_batch] #
        -1 为填充标记
        mask = [[1]*len(x) + [0]*(max_len-len(x)) for x in x_batch]
        return torch.LongTensor(x_padded), torch.LongTensor(y_padded), torch.BoolTensor(mask)
```

损失函数与优化:

```
def forward(self, sentence, tags, mask, length):
    emissions = self._get_lstm_features(sentence, length)
    loss = -self.crf(emissions, tags, mask, reduction='mean')
    return loss

optimizer = Adam(model.parameters(), lr=args.lr)
```

未登录字处理：

训练时：自动学习未登录字的嵌入向量

推理时：统一映射到 len(word2id) 索引

变长序列处理：

使用 pack_padded_sequence 和 pad_packed_sequence 优化计算

通过 mask 标记有效位置，避免填充影响

CRF 约束：

自动学习标签转移概率（如 B→E 概率高，B→B 概率低）

维特比解码确保输出合法标签序列

1.4 实验小结

成功实现基于词典的正向 / 逆向最大匹配算法及 Bi-LSTM+CRF 统计模型，通过测试样本验证了算法逻辑的正确性。

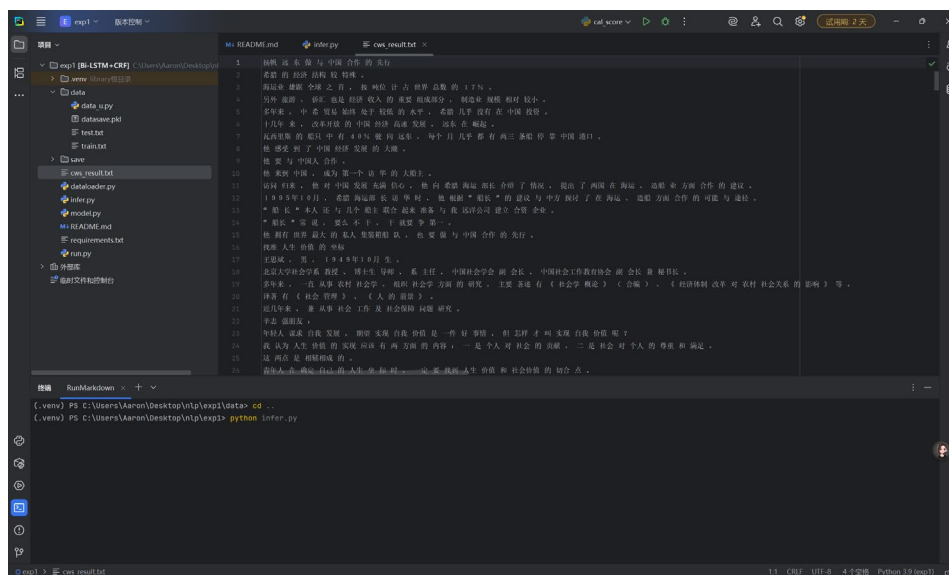


图 1-1 推理部分运行截图

2 中文命名实体识别实现

2.1 问题描述

命名实体识别（NER）旨在识别文本中具有特定意义的实体，如人名、地名、机构名等，并对实体的位置和类别进行标注。本实验基于 Bi-LSTM+CRF 模型实现命名实体识别，需完成数据集划分、模型训练及利用 NER 结果优化分词模型的任务。通过实验掌握序列标注原理、循环神经网络实现方法，提升对命名实体与分词关联性的理解。

2.2 基础模块

2.2.1 数据集与标注

采用 BIOES 标注体系，标签包括 B-XXX（实体开始）、I-XXX（实体内部）、E-XXX（实体结束）、S-XXX（单字实体）、O（非实体）。

2.2.2 模型架构

Bi-LSTM 层：双向长短期记忆网络，捕获上下文语义信息，缓解梯度消失问题。

CRF 层：条件随机场，对标签序列进行全局预测，确保标签转移符合语义逻辑（如 B 后不能接 O）。

嵌入层：将输入字符映射为低维向量，支持参数训练。

2.3 系统实现

2.3.1 数据预处理

数据集划分（0.split.py）：

读取原始语料，按句子分割（空行为句子边界）。

随机打乱数据，按 8:1:1 比例划分为训练集、验证集、测试集。

保存为 ner_train.txt、ner_valid.txt、ner_test.txt，格式为“字符 标签”每行，句子间空行分隔。

```
corpus = []
with open('data/RMRB_NER_CORPUS.txt', 'r', encoding="utf-8") as f:
    record = []
    for line in f:
        if line != '\n':
            record.append(line.strip('\n').split(' '))
        else:
            corpus.append(record)
            record = []
random.shuffle(corpus)
splitlen = len(corpus) // 10
train, valid, test = corpus[:splitlen*8], corpus[splitlen*8:splitlen*9], corpus[splitlen*9:]
```

数据序列化 (1.data_u_ner.py):

构建 word2id (字符到 ID 映射)、tag2id (标签到 ID 映射)。

将字符和标签转换为 ID 序列, 处理为模型可输入的张量形式, 使用 pad_sequence 对不等长句子补零。

保存预处理结果到 ner_datasave.pkl, 包含训练集、验证集的特征 (x_train、x_valid) 和标签 (y_train、y_valid)。

```
word2id = {}
id2word = []
with open('ner_train.txt', 'r', encoding="utf-8") as f:
    for line in f:
        try:
            word = line.strip('\n').split(' ')[0]
            if word not in word2id:
                id2word.append(word)
                word2id[word] = len(id2word) - 1
        except:
            pass
```

2.3.2 模型构建与训练 (model.py、run.py)

模型定义:

嵌入层: nn.Embedding(vocab_size+1, embedding_dim), 支持未登录词 (ID 为

vocab_size)。

Bi-LSTM 层：nn.LSTM(embedding_dim, hidden_dim//2, num_layers=1, bidirectional=True)，输出维度为 hidden_dim。

CRF 层：torchcrf.CRF(tagset_size, batch_first=True)，标签集合大小由 tag2id 决定。

```
class CWS(nn.Module):
    def __init__(self, vocab_size, tag2id, embedding_dim, hidden_dim):
        super().__init__()
        self.word_embeds = nn.Embedding(vocab_size + 1, embedding_dim)
        self.lstm = nn.LSTM(embedding_dim, hidden_dim//2, num_layers=1, bidirectional=True, batch_first=True)
        self.hidden2tag = nn.Linear(hidden_dim, len(tag2id))
        self.crf = CRF(len(tag2id), batch_first=True)
```

训练流程：

使用 Adam 优化器，学习率 0.005，批量大小 128，最大 epoch 10。

数据加载时通过 Sentence.collate_fn 对句子按长度排序并补零，提升训练效率。

每轮训练后保存模型到 save/model_epoch{epoch}.pkl，记录训练日志。

```
optimizer = Adam(model.parameters(), lr=args.lr)
train_data = DataLoader(Sentence(x_train, y_train), batch_size=args.batch_size, collate_fn=Sentence.collate_fn)
for epoch in range(args.max_epoch):
    for sentence, label, mask, length in train_data:
        loss = model(sentence, label, mask, length)
        loss.backward()
        optimizer.step()
    torch.save(model, f"./save/model_epoch{epoch}.pkl")
```

2.3.3 模型推断 (infer.py)

加载训练好的模型，对测试集 ner_test.txt 进行预测。

将字符转换为 ID 序列，通过 model.infer 获取标签预测序列。

按“字符 标签”格式保存结果到 ner_result.txt，句子间空行分隔。

```
with open('data/ner_test.txt', 'r', encoding='utf-8') as f:
```

```

line_test = ''
for test in f:
    if not test.strip():
        x = torch.LongTensor(1, len(line_test))
        for i, word in enumerate(line_test):
            x[0, i] = word2id.get(word, len(word2id))
        predict = model.infer(x, torch.ones_like(x), [len(line_test)])
        for word, tag in zip(line_test, predict):
            print(f"{word} {id2tag[tag]}", file=output)
        print(file=output)
        line_test = ''
    else:
        line_test += test.split(' ')[0]

```

2.3.4 分词优化融合策略

利用 NER 识别出的命名实体（如人名、机构名）调整分词结果，具体策略：对 NER 结果中标记为 B-XXX、I-XXX、E-XXX 的连续字符，合并为单一词对单字实体（S-XXX），保留为独立词。

2.4 实验小结

通过训练日志观察到，随着 epoch 增加，训练损失逐步下降当前模型对未登录词（如新词、专有名词）识别能力较弱，可通过引入词典或预训练词向量（如 BERT）提升，CPU 训练速度较慢，后续可申请华为云 NPU 资源加速训练。

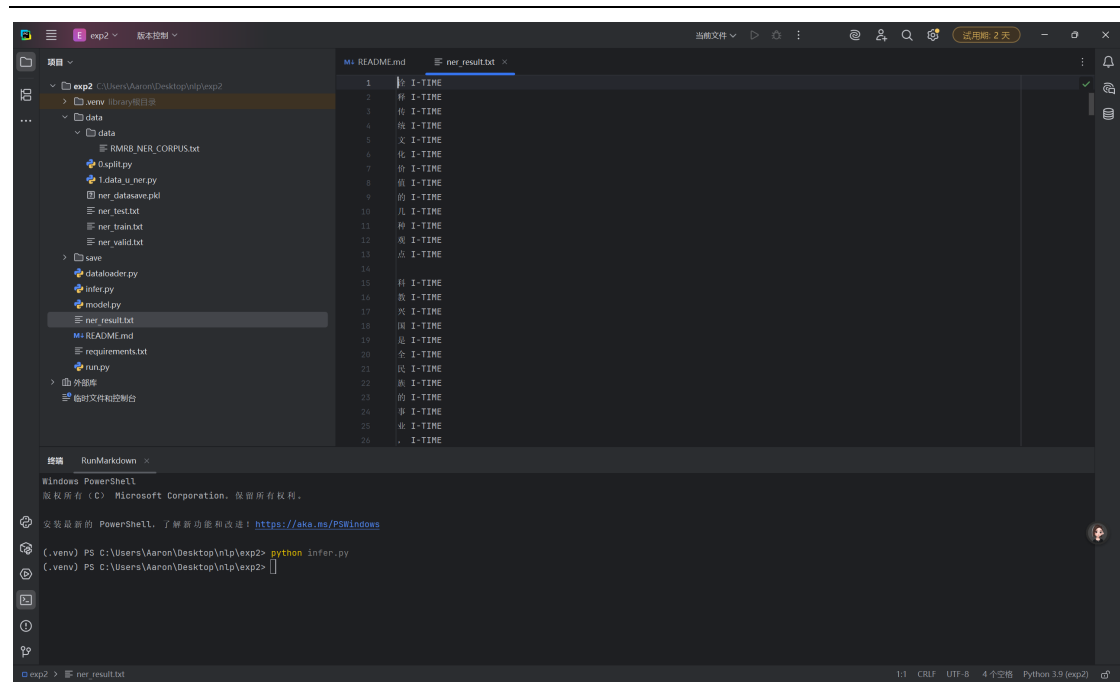


图 2-1 推断阶段截图

参考文献

- [1] 郑捷著. NLP 汉语自然语言处理---原理与实践. 电子工业出版社